Laboratorul 3

Operațiile de backup și restore.

Instrucțiuni DML si DQL

Laborator 3 – Operatia de backup si restore. Instructiuni DML si DQL

Obiective

1. Sa realizăm operația de backup a unei baze de date.
2. Sa realizăm operatia de restore a bazei de date.
3. Sa definim instructiuni de DML pentru popularea tabelelor din BD Companie.
4. Sa definim instructiuni de DQL pentru interogarea tabelelor din baza de date.

Noțiuni teoretice

I Operatiile de backup si restore

Componenta SQL Server de backup si restore furnizeaza o garantie esentiala pentru protejarea datelor critice stocate in bazele de date SQL.

I.1 Backup

Implementarea unei strategii de backup si restore protejeaza baza de date impotriva pierderii de date care pot aparea in cazul unor defectiuni la nivel de sistem sau de baza de date.

Backup-ul este o copie a datelor care este folosita pentru a restaura si a recupera datele dupa o posibila defectiune a sistemului. Backup-ul permite restaurarea datelor dupa un fisier creat la operatia efectiva de backup. Suplimentar, backup-ul bazei de date este folositor si in scopuri de rutina, cum ar fi copierea datelor de la un server la altul etc.

Backup-ul poate fi facut complet (full) sau diferential (differential).

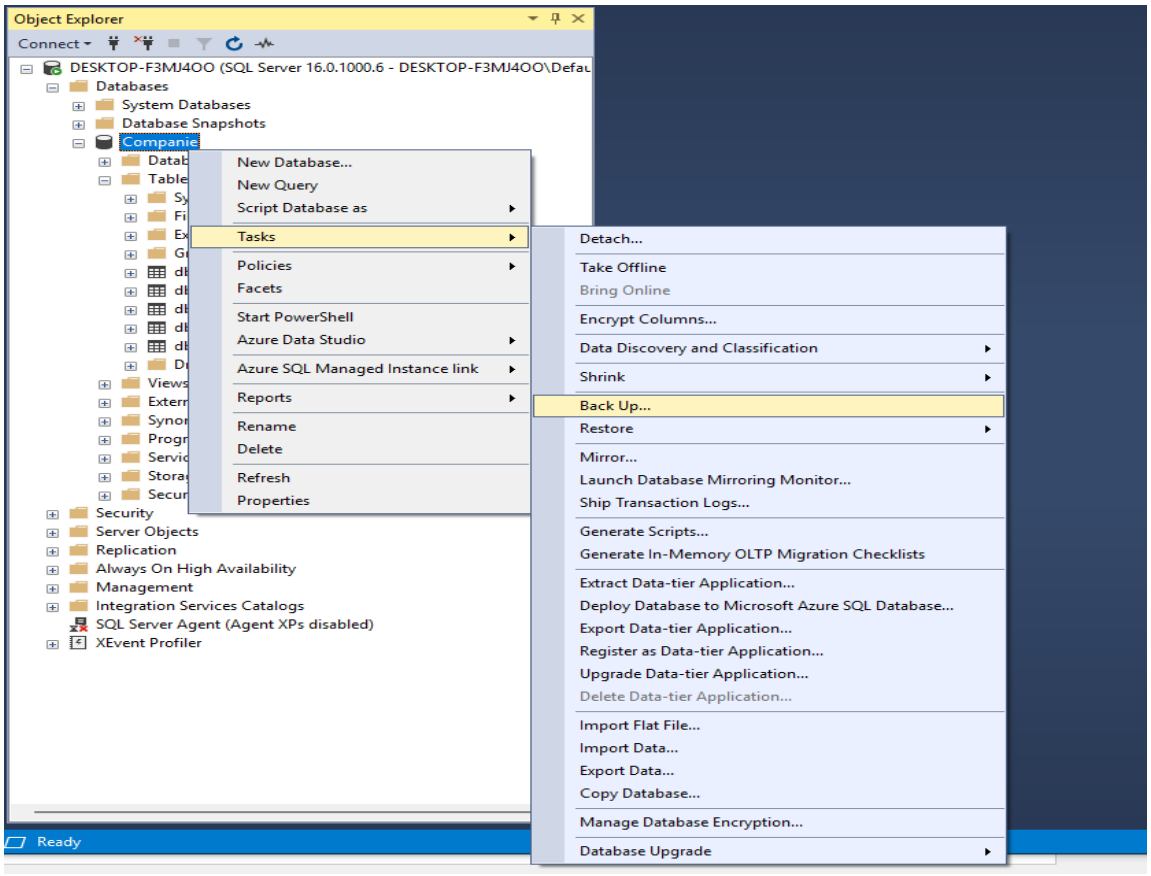
Exista 3 categorii de backup:

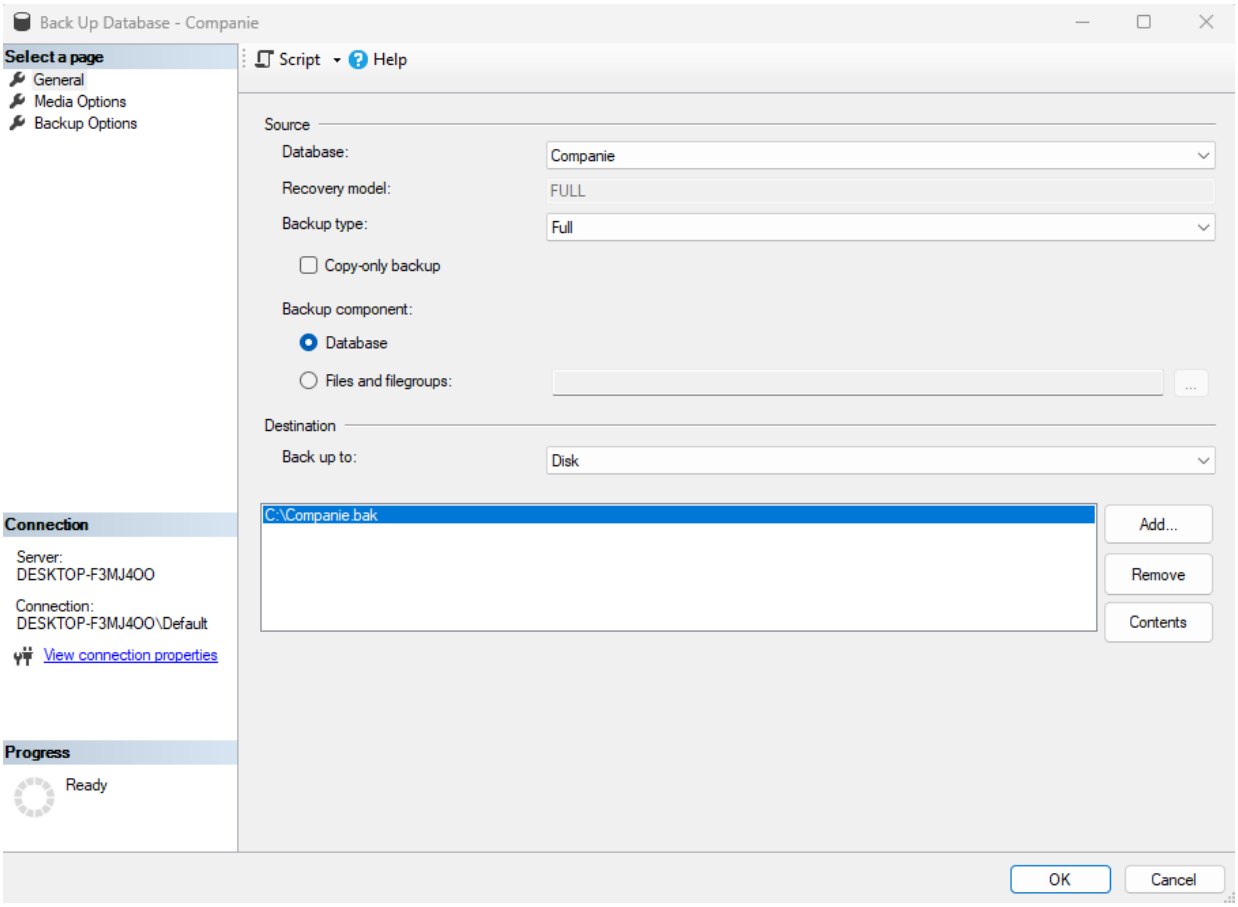
- **Data backup** - acest backup este orice backup care include intreaga imagine a unuia sau a mai multor fisiere de date. Un backup de date, salveaza toate datele si suficiente loguri pentru a permite restaurarea ulterioara a datelor. Backup-urile de date pot fi facute complet sau partial pentru o baza de date, sau pentru unul sau mai multe fisiere.
- **Backup diferential** - este bazat pe un backup de date facut anterior, cunoscut sub numele de base backup (backup de baza). Fiecare tip major de backup de date are un corespondent diferential. Backup-ul de baza este cel mai recent backup facut fie complet, fie partial. Un backup diferential contine numai date care au fost modificate sau adaugate de la base backup. Inainte de a restaura un backup diferential, trebuie restaurata baza (base).
- **Backup transaction log-uri** (numit si log backup) - include toate recordurile log care nu au fost pastrate in backup-ul de log anterior.

Pasii pentru a face un backup full sunt:

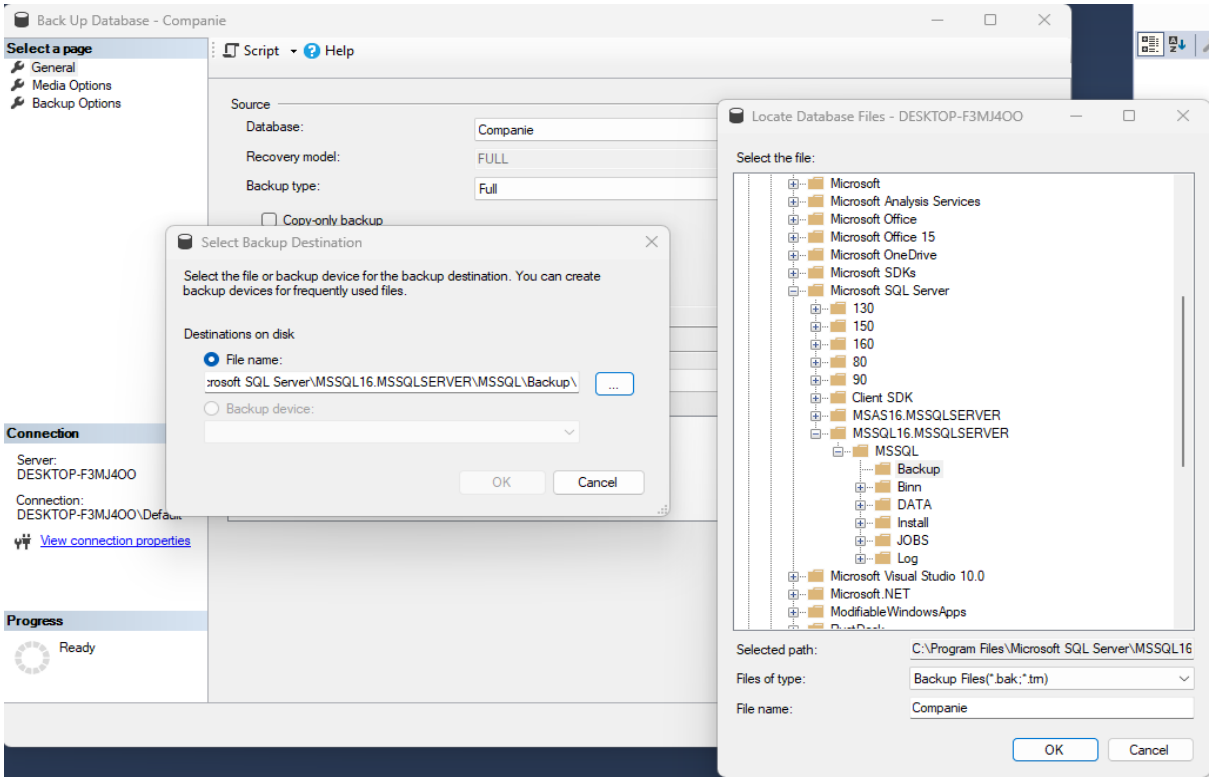
- se apasa click dreapta pe baza de date si se selecteaza optiunea *Task*
- se apasa *Backup*

Laborator 3 – Operatia de backup si restore. Instructiuni DML si DQL





Daca se doreste crearea fisierului de backup in alt loc pe harddisk decat cel implicit, se apasa *remove* partea de jos a ferestrei. Se apasa apoi *Add* si se alege noua locatie, precum si numele fisierului de backup (Companie.bak).

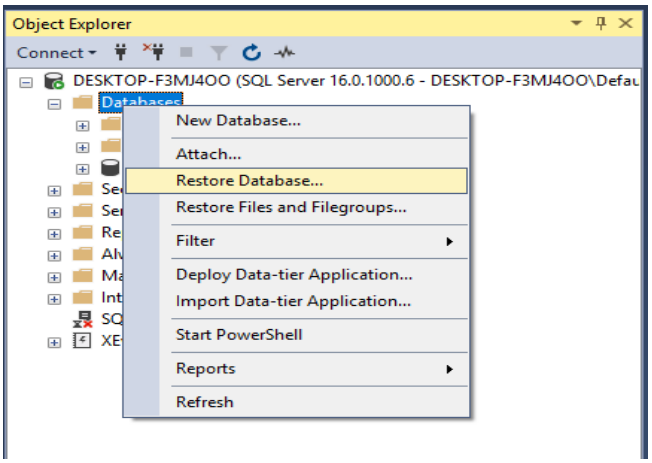


- odata aparuta locatia in acea fereastra, se da click pe ea si apoi se apasa *Next*
- apare un mesaj de realizare cu succes a backup-ului

I.2 Restore

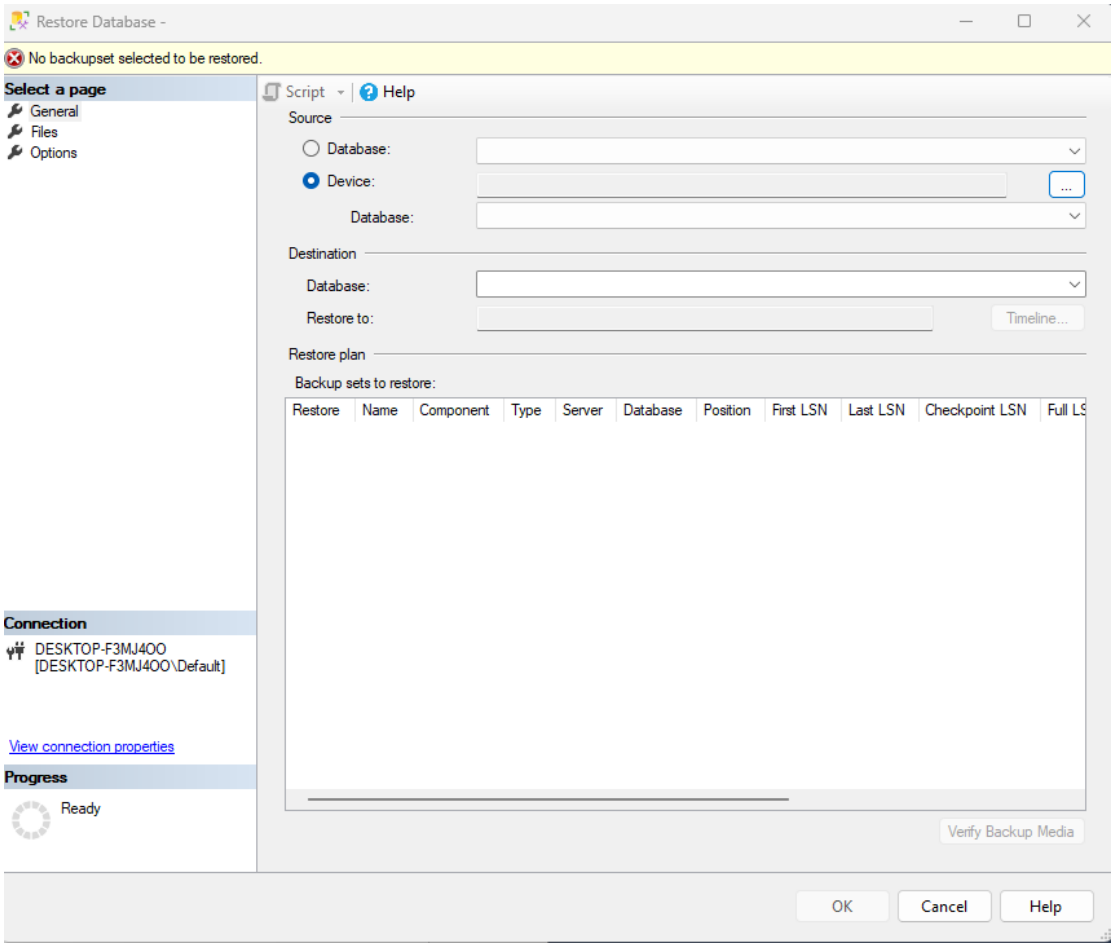
Operatia inversa a backup-ului este restore, ea realizandu-se asemanator.
Pentru a realiza operatia de Restore, pasii sunt urmatorii:

- in fereastra *Object Explorer*, se da click dreapta pe *Databases* > *Restore Database.*

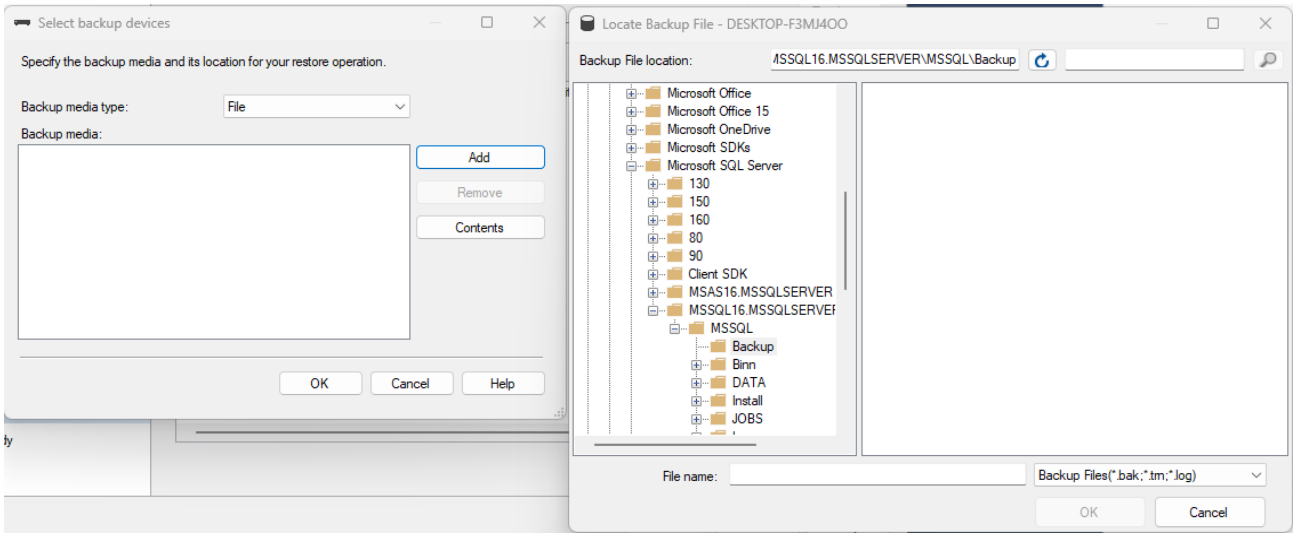


- se va deschide fereastra *Restore Database.*

- in zona *Source*, se selecteaza *Device* si click pe cele 3 puncte din dreptul *Device* pentru a selecta locatia de unde este salvat fisierul de backup.



- in noua fereastra deschisa, *Select backup devices*, se apasa butonul *Add* pentru a selecta locatia fisierului de backup ce va fi folosit.



II Componenta DML

SQL furnizeaza trei comenzi care permit consultarea, modificarea si actualizarea continutului bazei de date. Actualizarea se refera la adaugarea unor noi randuri intr-un tabel (**INSERT),** la modificarea uneia sau mai multor valori dintr-un rand (**UPDATE)** si la stergerea unuia sau mai multor randuri dintr-un tabel (**DELETE**).

II.1 Comanda INSERT

Clauza INSERT se foloseste pentru a insera randuri intr-un tabel.

```
INSERT INTO tabel [(col1, col2, …)]
VALUES (expresie 1, expresie2, …);
```

Intre tipul de date al coloanei si expresia din clauza VALUES trebuie sa fie corespondenta pozitionala. Daca nu se specifica lista de coloane in VALUES, trebuie precizate atatea valori corespondente pozitional cate campuri are tabela. Pot fi specificate pentru campuri si valorile NULL, respectiv DEFAULT.

Exemplu:

Inserarea in tabelul *Departamente* a unui nou departament cu codul 40, numele IT si sediul in Bucuresti:

```
INSERT INTO Departamente (Cod_departament, Nume_departament, Locatie)
VALUES (40, IT, ‘Bucuresti’);

INSERT INTO Deptartamente
VALUES (40, IT, ‘Bucuresti’);
```

II.2 Comanda UPDATE

Prin clauza UPDATE pot fi modificate randurile existente intr-un tabel. Sintaxa pentru clauza UPDATE:

```
UPDATE nume_tabela
SET coloana_1 = expresie_1,
coloana_2 = expresie_2,
….
coloana_n = expresie_n
WHERE conditie
```

Este obligatorie mentionarea numelui tabelei target in clauza UPDATE. Daca se doreste filtrarea unui anumit set de randuri, atunci va fi folosita clauza WHERE, iar rezultatul UPDATE-ului o sa afecteze doar randurile care indeplinesc conditia din WHERE.

In cazul in care un UPDATE este folosit fara WHERE, toate randurile din tabela vor fi modificate.

Exemplu:

Se doreste modificarea locatiei departamentului 40.

```
UPDATE Departamente
SET locatie = 'Craiova'
WHERE cod_departament = 40;
```

II.3 Comanda DELETE

Prin clauza DELETE se poate efectua stergerea randurilor dintr-o tabela. Poate fi utilizata si clauza WHERE, astfel incat sa fie sterse doar anumite inregistrari.

Pentru stergerea tuturor inregistrarilor dintr-o tabela, poate fi utilizata si clauza TRUNCATE (care face parte din categoria DDL).

Sintaxa clauzei DELETE:

```
DELETE FROM nume_tabela
WHERE conditie
```

Exemple:

1. Sa se stearga din tabela Departamente toate departamentele care au locatia in Bucuresti.

```
DELETE FROM Departamente
WHERE locatie = 'Bucuresti'
```

2. Sa se stearga toate datele din tabela Departamente folosind DELETE si toate datele din tabela Angajati folosind TRUNCATE.

```
DELETE FROM Departamente
TRUNCATE TABLE Angajati
```

Diferente intre TRUNCATE si DELETE:

- DELETE permite stergerea doar anumitor informatii folosind clauza WHERE, pe cand TRUNCATE sterge toate datele din tabela.
- TRUNCATE este mai rapid decat DELETE, pentru ca DELETE trebuie sa logheze in tranzactii fiecare inregistrare stearsa, pe cand TRUNCATE foloseste o singura tranzactie pentru toate randurile sterse.
- DELETE poate utiliza JOIN-uri, pe cand TRUNCATE nu poate.
- DELETE necesita drepturi de stergere, pe cand TRUNCATE necesita drepturi de ALTER.

III Instructiuni DQL (Data Query Language)

Ordinea de scriere a unei interogari (query)

```
1.SELECT <coloane>
2.FROM <tabele>
3.WHERE <conditii>
4.GROUP BY <agregari>
5.HAVING <conditii pe agregari>
6.ORDER BY <ordonarea datelor>
```

Ordinea de rulare a unei interogari (query)

```
1.FROM <tabele>
2.WHERE <conditii>
3.GROUP BY <agregari>
4.HAVING <conditii pe agregari>
5.SELECT <coloane>
6.ORDER BY <ordonarea datelor>
```

Numai clauzele SELECT, FROM sunt obligatorii; celelalte depind de natura cererii.

III.1 Clauzele SELECT si FROM

III.1.1 Returnarea tuturor campurilor

Pentru a returna toate datele dintr-un tabel se utilizeaza cererea SQL cu sintaxa:

```
SELECT * FROM Tabel;
```

in care * este un simbol prin care se specifica dorinta de includere a ”tuturor campurilor” in rezultat. Campurile vor fi returnate in ordinea in care au fost create in baza de date.

Exemplu:

Sa se extraga toate datele din tabelul Angajati al bazei de date Companie:

```
SELECT * FROM Angajati;
```

III.1.2 Returnarea unui singur camp

Pentru a extage valorile aferente unui singur camp dintr-un tabel se foloseste sintaxa:

```
SELECT Camp FROM Tabel;
```

Exemplu:

Sa se extraga numele tuturor angajatilor din baza de date Companie:

```
SELECT Nume FROM Angajati;
```

III.1.3. Returnarea mai multor campuri

Pentru a extage valorile aferente mai multor campuri dintr-un tabel se foloseste sintaxa:

```
SELECT Camp1, Camp2,...Camp n FROM Tabel;
```

Campurile vor fi returnare in ordinea specificata.

Exemplu:

Sa se extraga numele, prenumele si data angajarii tuturor angajatilor din baza de date Companie:

```
SELECT Nume, Prenume, DataAngarii FROM Angajati;
```

Folosirea de aliasuri

Pana acum, a fost exemplificata returnarea anumitor campuri dintr-un tabel si stabilirea ordinii in care acestea sa fie returnate. In plus, campurile returnate pot fi redenumite folosind asa numitele *aliasuri*. Aliasuri pot fi stabilite atat pentru tabele, cat si pentru campuri.

Pentru a defini un alias se foloseste cuvantul cheie AS, avand la dreapta numele campului (tabelei) si la stanga numele alias asociat:

```
SELECT CampA AS A, CampB AS B, CampC AS C
FROM Tabel AS T;
```

Prin declaratia de mai sus vor fi redenumite campurile tabelului Tabel astfel: CampA in A, CampB in B si CampC in C. In momentul in care va fi returnat rezultatul acestei interogari, aceste aliasuri vor aparea in locul numelor campurilor din baza de date.

Daca se doreste definirea unui alias si pentru tabel, poate fi folosita sintaxa de mai jos. In cazul cererilor la o singura tabela definirea de alias la tabela nu este justificata.

```
SELECT T.CampA AS A, T.CampB AS B, T.CampC AS C
FROM Tabel AS T;
```

Observatie: Specificatiile ANSI pentru SQL-99 permit lipsa cuvantului cheie AS. Un alias are efect local doar in cererea in care a fost definit:

```
SELECT T.CampA A, T.CampB B, T.CampC C
FROM Tabel T;
```

Exemplu:

Sa se afiseze numele si prenumele angajatilor din baza de date Companie.

```
SELECT Nume AS NumeAngajat, Prenume AS PrenumeAngajat

FROM Angajati;
```

Rezultatul acestei interogari va avea urmatoarea forma:

Results | Messages

	NumeAngajat	PrenumeAngajat
1	Popescu	Maria
2	Ionescu	Marian
3	Udrea	Ioana
4	Lungu	Ioan
5	Cristea	Dan
6	Ionescu	Andrei
7	Vasile	Ion
8	Udrea	Elena
9	Aldea	Eliza

Returnarea campurilor calculate

Pana acum am vazut cum putem extrage informatiile din campurile bazei de date si cum le putem redenumi. Este posibil insa ca pornind de la informatiile continute in campurile bazei de date, in interogarile SELECT sa fie returnate valori obtinute din campuri calculate. Pot aparea doua situatii de acest gen:

- Campuri calculate cu operatori matematici

- Operatii cu siruri de caractere

- Campuri calculate cu operatori matematici

Presupunand ca avem un tabel Produse care contine printre altele campurile cu informatii despre NumeProdus, Cantitate si PretUnitar, putem realiza o cererea care sa returneze informatiile:

```
SELECT NumeProdus, Cantitate, Cantitate * PretUnitar AS ValoareTotala

FROM Produse;
```

In rezultat a fost creat un camp ValoareTotala, a carei valoare este calculata pe baza valorilor din campurile Cantitate si PretUnitar (inmultire) la foecare inregistrare din tabelul Produse.

- Concatenarea sirurilor de caractere

SQL Server foloseste simbolul + pentru concatenarea sirurilor de caractere:

```
SELECT Nume + ' ' + Prenume AS NumeComplet FROM Angajati;
```

Interogarea de mai sus va returna o singura coloana, denumita NumeComplet, care va continue informatia concatenata a campurilor Nume si Prenume din tabelul Angajati al bazei de date Companie.

3.2 Ordonarea datelor – clauza ORDER BY

In baza de date, inregistrarile sunt stocate intr-o ordine arbitrara. Clauza ORDER BY sorteaz inregistrarile rezultate intr-o interogare, in functie de unul sau mai multe campuri specificate, in ordine crescatoare sau descrescatoare.

Implicit, ordinea de sortare este cea crescatoare (de la a la z, respectiv A la Z, de la 0 la 9).

Sintaxa este urmatoarea:

```
SELECT CampA, CampB, CampC

FROM Tabel
```

[ORDER BY CampA[ASC|DESC], CampB[ASC|DESC]…];

Clauza ORDER BY este optionala si, de obicei, este ultima clauza a unei interogari SQL.

Exemplu : In ambele exemple de mai jos, numele angajatilor sunt sortate in ordine crescatoare:

```
SELECT Nume, Prenume

FROM Angajati

ORDER BY Nume;
```

```
SELECT Nume, Prenume

FROM Angajati

ORDER BY Nume ASC;
```

Pentru a sorta in ordine descrescatoare (de la Z la A, de la 9 la 0), trebuie adaugat cuvantul cheie DESC dupa campul care dorim sa fie sortat in ordine descrescatoare.

Exemplu : Urmatorul exemplu selecteaza salariile angajatilor sorteaza inregistrarile returnate in ordine descrescatoare:

```
SELECT Nume, Salariu

FROM Angajati

ORDER BY Salariu DESC, Nume;
```

Inregistrarile sunt ordonate mai intai dupa primul camp specificat in clauza ORDER BY.

Inregistrarile care au valori egale pentru acel acel camp sunt sortate apoi dupa valorile din al doilea camp specificat, si tot asa.

3.3 Clauza WHERE

Clauza WHERE permite filtrarea rezultatului unei interogari in functie de anumite conditii.

Sintaxa este urmatoarea :

```
SELECT CampA, CampB

FROM Tabel

WHERE conditii;
```

In *conditii* se pot specifica egalitati sau inegalitati pentru valorile anumitor campuri, se poate verifica ca un camp se incadreaza intr-un anumit interval, etc.

- Conditii de egalitate

O astfel de conditie verifica daca valoarea unui camp este egala cu o constanta sau valorile a doua campuri sunt egale. Pot fi utilizate expresii de calcul cu valorile campurilor tabelei. O sintaxa uzuala este:

SELECT CampA, CampB

FROM Tabel

WHERE CampC = valoare;

Inregistrarile returnate vor fi doar cele care au CampC egal cu valoare. In acest caz, conditia de egalitate va fi evaluate ca fiind TRUE. Daca este evaluata FALSE sau UNKNOWN (datorita unei valori NULL), atunci inregistrarea respectiva nu se va regasi in rezultatul interogarii.

Exemple:

1. Selectam o singura inregistrare din tabelul Angajati, corespunzatoare angajatului cu ID-ul 5.

SELECT Nume, Prenume

FROM Angajati

WHERE AngajatID = 5;

2. Selectam toti angajatii care au numele de familie Ionescu:

SELECT Nume, Prenume

FROM Angajati

WHERE Nume = 'Ionescu';

- Conditii de inegalitate

Implica un operator relational intre valoare aunui camp sau expresie si o constanta sau expresie.

Sintaxa:

SELECT CampA, CampB

FROM Tabel

WHERE CampC >|<|>=|<= valoare;

Inregistrarile returnate vor fi doar cele pentru care conditia din clauza WHERE este evaluata ca fiind TRUE. Daca este evaluata FALSE sau UNKNOWN (datorita unei valori NULL), atunci inregistrarea respectiva nu se va regasi in rezultatul interogarii.

Exemplu : Selectam toti angajatii care au salariul mai mare de 2500 ron.

```
SELECT Nume, Prenume

FROM Angajati

WHERE Salariu > 2500;
```

Obs: Campul/campurile din clauza WHERE nu trebuie neaparat sa se regaseasca si in lista de campuri returnate, din clauza SELECT.

Combinarea conditiilor

In clauza WHERE pot aparea expresii obtinute ca si combinatii de conditii, cu operatorii logici AND si OR sau chiar negarea unei conditii (NOT), cu sintaxa:

```
SELECT CampA, CampB

FROM Tabel

WHERE Conditie1 AND|OR Conditie2;
```

Exemple:

1. Selectam angajatii care poarte numele Ionescu sau Popescu.

```
SELECT Nume, Prenume

FROM Angajati

WHERE Nume = ‘Ionescu’ OR Nume = ‘Popescu’;
```

2. Selectam angajatii care au numele Ionescu si prenumele Andrei.

```
SELECT Nume, Prenume

FROM Angajati

WHERE Nume = ‘Ionescu’ AND Prenume = ‘Andrei;
```

3. Selectam angajatii care au salariul cuprins intre 2500 si 5000 ron.

```
SELECT Nume, Prenume

FROM Angajati
```

WHERE Salariu >= 2500 AND Salariu <= 5000;

3.4 Operatori SQL simpli

DISTINCT

Se utilizeaza atunci cand se doreste eliminarea duplicatelor in rezultat. Astfel, sunt omis inregistrarile ce contin date aceleasi date in campurile selectate. Sunt incluse in rezultatele interogarii numai acele inregistrari pentru care valorile fiecarui camp listat in instructiunea SELECT este unica.

Sintaxa uzuala:

SELECT DISTINCT Camp

FROM Tabel

Exemplu : Mai multi angajati din tabelul Angajati pot avea acelasi nume de familie. Daca doua inregistrari contin Ionescu in campul Nume, urmatoarea instructiune SQL returneaza o singura inregistrare ce contine informatia Ionescu:

SELECT DISTINCT Nume

FROM Angajati;

Fara cuvantul cheie DISTINCT, interogarea ar returna toate inregistrarile aferente angajatilor cu numele Ionescu.

In cazul in care clauza SELECT contine mai multe campuri, combinatia valorilor din toate campurile trebuie sa fie unica pentru ca o inregistrare data sa fie inclusa in rezultat.

BETWEEN

Este un operator prin care se determina daca valoarea unei expresii se gaseste intr-un interval de valori specificat, limitat de valoarea limita inferioara si valoarea limita superioara

Sintaxa:

SELECT CampA, CampB

FROM Tabel

WHERE CampC BETWEEN LimitaInferioara AND LimitaSuperioara;

Exemple:

1. Selectam angajatii care au salariul cuprins intre 2500 si 5000 ron.

```
SELECT Nume, Prenume
FROM Angajati
WHERE Salariu BETWEEN 2500 AND 5000;
```

2. Selectam angajatii care au numele cuprins intre Ionescu si Popescu.

```
SELECT Nume, Prenume
FROM Angajati
WHERE Nume BETWEEN ‘Ionescu’ AND ‘Popescu;
```

LIKE

Acest operator se utilizeaza in conjunctie cu valori de tip sir de caractere si compara o expresie de acest tip cu un sablon intr-o expresie SQL.

Sintaxa este urmatoarea:

```
SELECT CampA, CampB
FROM Tabel
WHERE CampC LIKE sablon
```

Pentru sablon se poate specifica o valoare completa (‘Ionscu’ sau se poate utiliza caracterul % pentru a gasi un interval de valori (‘Ion%’).

Exemple :

1. Selectam toti angajatii al caror nume incepe cu ‘Ion’.

```
SELECT Nume, Prenume
FROM Angajati
WHERE Nme LIKE 'Ion%'
```

2. Selectam toti angajatii al caror nume se termina cu ‘u’

```
SELECT Nume, Prenume
FROM Angajati
WHERE Nume LIKE '%u'
```

Observatie: Caracterul % inlocuieste un sir de caractere de orice lungime.

Top-N

Uneori nu dorim sa vedem toate inregistrarile dintr-o baza de date care corespund unor criterii, ci doar pe primele N. In situatii de acest gen, in SQL Server avem la dispozitie cuvantul cheie TOP pentru a specifica numarul de inregistrari din rezultat. Sintaxa unei astfel de interogari arata astfel:

SELECT TOP NrInreg CampA, CampB

FROM Tabel;

NrInreg specifica, functie de format, numarul inregistrarilor ce se doresc in rezultat. Astfel, poate specifica numarul inregistrarilor returnate sau atunci cand este urmat de cuvantul cheie PERCENT, se specifica procentul din numarul total de inregistrari care va fi returnat.

Exemple*:*

1. Selectam primele 5 inregistrari din tabelul Angajati

SELECT TOP 5 AngajatID, Nume

FROM Angajati

2. Selectam cei 10% angajati din tabelul Angajati care au salariul cel mai mic

SELECT TOP 10 PERCENT AngajatID, Salariu

FROM Angajati

ORDER BY Salariu;

IV Exercitii obligatorii

1. Selectati toate datele despre departamentele din tabelul Departamente.

2. Selectati departamentele al caror nume incepe cu ‘I’.

3. Selectati numele si prenumele angajatilor care au fost angajati inainte de 01.01.2007 sau dupa

01.01.2008.

4. Selectati numele proiectelor coordonate de departementul cu ID-ul 2, ordonate descrescator.

5. Selectati proiectele al caror nume fie incepe cu ’A’, fie contine grupul de litere ’ea’ in

interiorul denumirii.

6. Selectati angajatii care au fost angajati dupa 01.03.2007 si care au salariul mai mare de 3000

ron.

7. Selectati angajatii care sunt afiliati departamentului cu ID-ul 1, ordonati crescator dupa nume.

8. Selectati numele, prenumele si salariul pentru angajatul cu cnp 1111111111111.

9. Selectati salariul, eliminand duplicatele.

10. Selectati numele si prenumele tuturor angajatilor din departamentul cu ID-ul 2.

11. Selectati numele, prenumele si salariul angajatilor cu salariu mai mare de 300.

12. Afisati o lista in care sa apara numele, prenumele si data nasterii pt toti angajatii.

13. Mariti salariul angajatilor cu 10% pt angajatii de la departamentul cu idul 1.

14. Stergeti toate informatiile despre intretinutii care au sex masculin.

V Exercitii suplimentare

1. Sa se afiseze ID-urile angajatilor care au persoane in intretinere nascute incepand cu anul 2000.
2. Sa se afiseze prenumele, numele si data angajarii pentru angajatii al caror prenume incepe cu "Maria" sau "Ioan", ordonati crescator dupa nume si prenume.
3. Sa se afiseze numele si prenumele persoanelor aflate in intretinere nascute in perioada 2001-2004, ordonate crescator dupa prenume si nume.
4. Sa se afiseze numele complet, judetul si orasul pentru angajatii barbati, ordonati crescator dupa nume si descrescător dupa data nasterii. Numele complet este de forma "Prenume Nume".
5. Sa se afiseze prenumele, numele și adresa completa pentru toti angajatii nascuti incepand cu anul 1980 care sunt coordonati de angajatul cu ID-ul 2. Adresa completa este de forma "Str, Nr, Oras, Judet".
6. Sa se afiseze numele, prenumele, salariul brut si salariul net al tuturor angajatilor, ordonati descrescator dupa salariul net si crescator dupa nume si prenume. Se cunoaste ca in tabela angajati se retine salariul brut, iar impozitul este 41,5%.
7. Sa se afiseze numele, prenumele si CNP-ul pentru primele 3 angajate cu cele mai mari salarii.
8. Sa se afiseze numele, prenumele si data nasterii primilor 10% dintre cei mai vechi angajati ai departamentului cu ID-ul 1 care sunt coordonati de angajatul cu ID-ul 1 sau de angajatul cu ID-ul 3.
9. Sa se afiseze numele, prenumele si salariul pentru cel mai tanar dintre angajatii al caror nume se termina in 'escu' sau care au fost angajati in perioada iunie 2004 - februarie 2007.

VI Crearea bazei de date Training, a tabelelor si a constrangerilor utilizand DDL

Exemplu: Se va considera ca business o companie de turism.
Fluxul acțiunilor:

- un client suna departamentul Call Center pentru a primi o oferta de concediu => Clientul este stocat in tabela Clienti
- Agentul din Call Center creeaza o rezervare pentru propunerea catre client => Rezervarea este creata si stocata in tabela Orders
- Clientul confirma sau nu rezervarea. Daca va confirma rezervarea => Rezervarea confirmata este stocata in tabela Orders_details
- Daca se confirma rezervarea, atunci se stiu detaliile despre rezervare => tabela Hotels, tabela Cities, tabela Countries

Sintaxa creare baza de date

CREATE DATABASE Reservations;
Conform exemplului de business de mai sus, se pot identifica urmatorele tabele: Customers, Countries , Cities, Hotels, Orders, Orders_details.

Creare tabele

Pentru a defini un tabel intr-o baza de date sunt necesare urmatoarele:

- Nume tabel
- Coloanele din tabel: nume coloane, tipuri de date pentru fiecare coloana, daca fiecare coloana admite sau nu NULL, primary key pentru fiecare tabela.

```
USE [Reservations]
GO

CREATE TABLE dbo.customers
(
	[idcustomer] [int] NOT NULL,
	[name] [nvarchar](50) NULL,
	[idcountry] [int] NOT NULL,
	CONSTRAINT [PK_customers] PRIMARY KEY (idcustomer)
)
```

Obs: *Coloanele idcustomer si idcountry nu accepta NULL. Cheia primara (PK) pentru tabelul Customers este idcustomer. Coloanele pentru care se va seta PK nu accepta NULL. Pentru un tabel se poate defini cel mult o cheie primara.*

In mod similar se creeaza si celelalte tabele mentionate mai sus.

```
USE Reservations
GO

CREATE TABLE dbo.cities
(
```

```
	cityid int NOT NULL,
	cityname nvarchar(50) NULL,
	idcountry int NOT NULL,
	CONSTRAINT PK_cities PRIMARY KEY (cityid)
)
GO

CREATE TABLE dbo.countries
(
	idcountry int NOT NULL,
	name nvarchar(255) NULL
	CONSTRAINT PK_countries PRIMARY KEY (idcountry)
)
GO

CREATE TABLE dbo.hotels
(
	idhotel int NOT NULL,
	name nvarchar(50) NULL,
	idcity int NOT NULL,
	stars int NULL,
	CONSTRAINT PK_hotels PRIMARY KEY (idhotel)
)
GO
CREATE TABLE dbo.orders (
	idorder int NOT NULL,
	idcustomer int NOT NULL,
	creationdate datetime NULL,
	CONSTRAINT PK_orders PRIMARY KEY (idorder)
)
GO
CREATE TABLE dbo.orders_details
(
	idorder int NOT NULL,
	totalprice money NULL,
	startdate datetime NULL,
	enddate datetime NULL,
	idhotel int NOT NULL,
	rooms int NULL,
	idtyperoom int NOT NULL,
	CONSTRAINT PK_orders_details PRIMARY KEY (idorder)
)
GO

CREATE TABLE dbo.type_room
(
	idtyperoom int NOT NULL,
	description nvarchar(50) NULL,
	CONSTRAINT PK_type_room PRIMARY KEY (idtyperoom)
)
```

Modificare tabele

Un tabel odata creat poate fi modificat utilizand comanda ALTER. In acest mod pot fi adaugate, sterse, modificate coloane.

Exemple:

1) Sa se adauge o coloana noua in tabelul Customers cu numele Address, de tip varchar(100).

alter table Reservations.dbo.customers
add Address varchar(100)

2) Sa se modifice tipul coloanei Address din tabelul Customers in varchar(150).

alter table Reservations.dbo.customers
alter column Address varchar(150)

3) Sa se stearga coloana Address din tabelul Customers.

alter table Reservations.dbo.customers
drop column Address

Adaugare constrangeri tabele existente

Exemple:

1) Sa se modifice tabela Customers astfel incat, campul Name sa nu accepte NULL.
alter table Reservations.dbo.customers
alter column Name nvarchar(50) NOT NULL

Obs: Daca tabelul Customers contine deja date, iar coloana Name are deja null-uri, atunci adaugarea constragerii NOT NULL nu va functiona. Mai intai se vor completa NULL-urile existente si apoi se va aplica constrangerea de NOT NULL pentru viitorii clienti inserati in tabela.

2) Sa se modifice tabela Hotels, astfel incat, campul idCity sa admita doar date unice.
alter table Reservations.dbo.Hotels
add constraint AC_City UNIQUE (idcity)

Obs: Adaugarea constrangerii de UNIQUE pentru IdCity inseamna din punct de vedere business ca 2 hoteluri nu pot fi din acelasi oras. De exemplu, un lant hotelier are un singur hotel in fiecare oras, Ramada ar putea fi un exemplu in acest sens.

3) Sa se stearga primary key-ul din tabela Cities. Sa se adauge primary key pe tabela Cities.
alter table Reservations.dbo.Cities
drop constraint PK_cities

alter table Reservations.dbo.Cities
add constraint PK_cities primary key (cityid)

4) Pentru fiecare tabela de mai sus sa se stabileasca Foreign key.
ALTER TABLE Orders
ADD FOREIGN KEY (idcustomer) REFERENCES Customers(idcustomer)

```
ALTER TABLE Orders_Details
ADD FOREIGN KEY (idorder) REFERENCES Orders(idorder)

ALTER TABLE Orders_Details
ADD FOREIGN KEY (idhotel) REFERENCES Hotels(idhotel)

ALTER TABLE Hotels
ADD FOREIGN KEY (idcity) REFERENCES Cities(cityid)

ALTER TABLE Cities
ADD FOREIGN KEY (idcountry) REFERENCES Countries(idcountry)

ALTER TABLE Customers
ADD FOREIGN KEY (idcountry) REFERENCES Countries(idcountry)
```

5) Sa se adauge constrangerea de CHECK astfel incat, in tabela Hotels, campul Stars sa nu contina valori mai mari de 5.

```
alter table Reservations.dbo.Hotels
add constraint Ch_stars CHECK (stars<=5)
```

6) Sa se modifice tabela Orders_details, astfel incat coloana rooms sa aiba valoarea default 1.

```
alter table Reservations.dbo.Orders_details
add constraint df_rooms default (1) for rooms
```

Creare tabele cu constrangeri

Exemple:
1) tabela Customers - campul Name sa nu accepte NULL.

```
CREATE TABLE dbo.customers1
(
	[idcustomer] [int] NOT NULL,
	[name] [nvarchar](50) NOT NULL,--1)tabela Customers-campul Name sa nu accepte NULL
	[idcountry] [int] NOT NULL,
	CONSTRAINT [PK_customers1] PRIMARY KEY (idcustomer)
)
```

2) tabela Hotels - campul idCity sa admita doar date unice.
3) constrangerea de CHECK in tabela Hotels, - campul Stars sa nu contina valori mai mari de 5.

```
CREATE TABLE dbo.hotels1
(
	idhotel int NOT NULL,
	name nvarchar(50) NULL,
	idcity int NOT NULL,
	stars int NULL,
	CONSTRAINT PK_hotels1 PRIMARY KEY (idhotel),
	CONSTRAINT UNK_hotels1 UNIQUE (idcity), -- 2) tabela Hotels - campul idCity sa admita doar date unice.
	CONSTRAINT Ch_hotels1 Check (stars<=5) -- 3) constrangerea de CHECK in tabela Hotels, - campul Stars sa nu contina valori mai mari de 5.
```

)

4) tabela Orders_details - coloana rooms sa aiba valoarea default 1.

```
CREATE TABLE dbo.orders_details1
(
	idorder int NOT NULL,
	totalprice money NULL,
	startdate datetime NULL,
	enddate datetime NULL,
	idhotel int NOT NULL,
	rooms int NULL default 1,
	idtyperoom int NOT NULL,
	stars int NULL,
	CONSTRAINT PK_orders_details1 PRIMARY KEY (idorder)
)
```